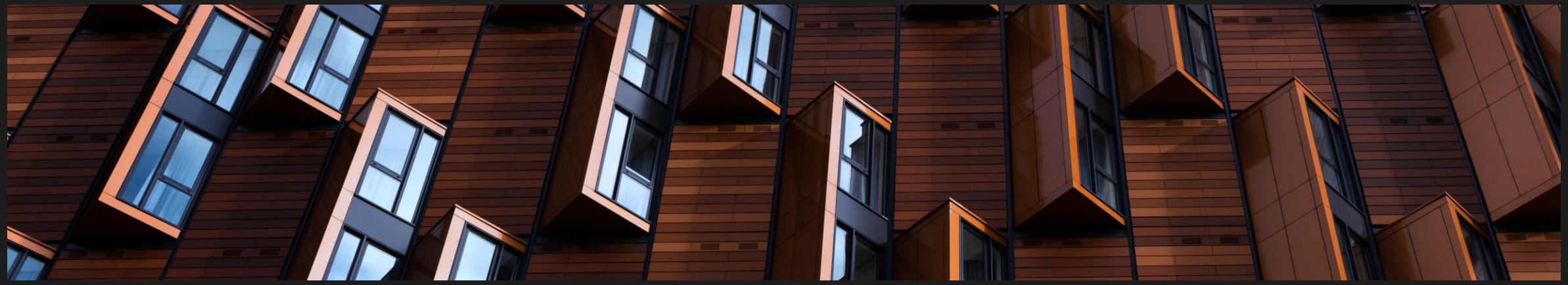
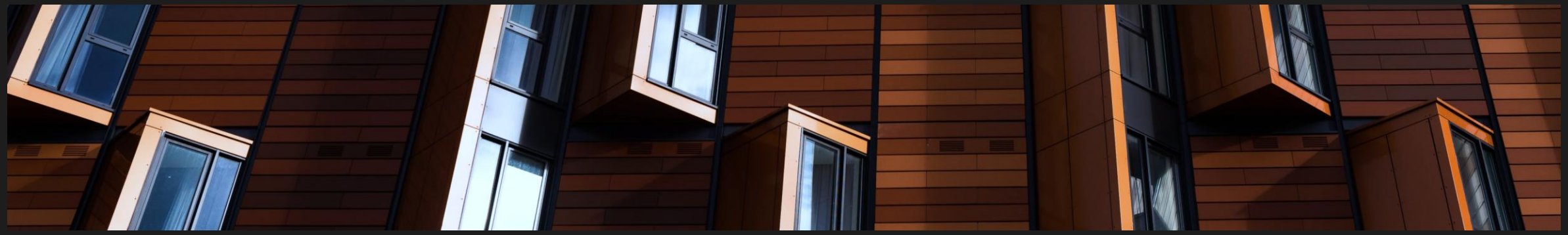


Kapitel 5

Entwurfsprinzipien & Clean Code



FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Agenda

Abschnitt 5.1

Entwurfsprinzipien

Abschnitt 5.2

Clean Code & Anti-Pattern

Abschnitt 5.3

Technische Schulden

Klausurrelevante Lernziele

- Kopplung & Kohäsion erläutern können
- Alle fünf SOLID-Prinzipien nennen & zwei davon erläutern können
- Drei Prinzipien von Clean Code Developer (CCD) nennen & erläutern können
- Notwendigkeit von Coding Conventions erläutern können
- Konzept der technischen Schulden erläutern können

Abschnitt 5.1

Entwurfsprinzipien

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Entwurfsprinzipien & kognitive Psychologie

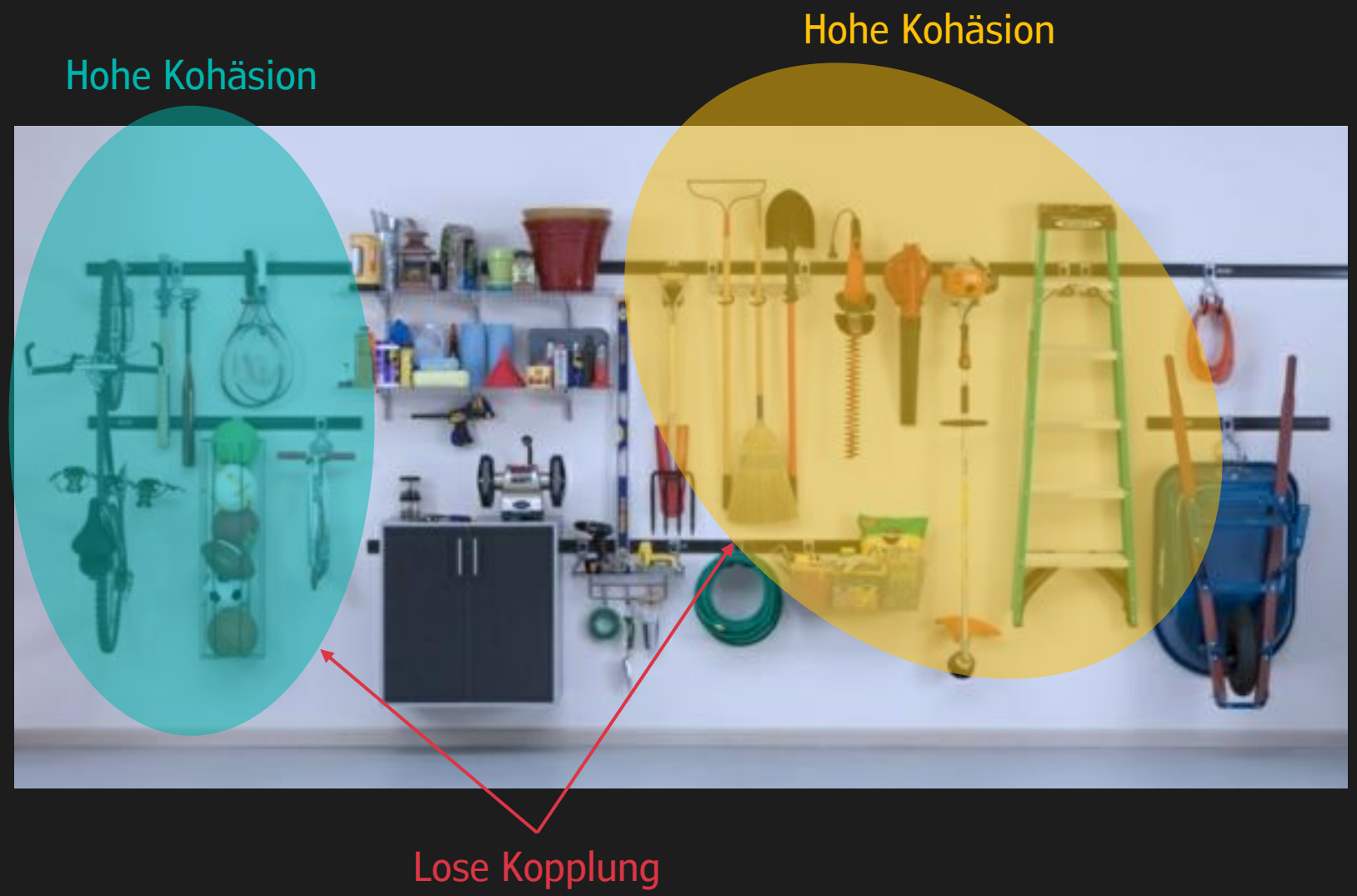
- Professionelle Softwareentwickler müssen im Allgemeinen sehr viel häufiger bestehenden Quellcode lesen & verstehen als neuen Quellcode schreiben
- Untersuchungen zeigen: Verstehen als Kombination aus Top-Down und Bottom-Up
- Kognitive Psychologie: drei strukturbildende Prozesse bei menschlichem Verstehen
  - Zusammenfassen zu sinnvollen Wissenseinheiten (Chunking)
  - Bildung von Hierarchien
  - Abstraktion wiederkehrender Muster als Schemata
- Entwurfsprinzipien fördern genau diese kognitiven Prozesse
  - Chunking: Modularität (Objekte, Bausteine etc.), Kopplung & Kohäsion, Information Hiding, Separtation of Concerns, explizite & minimale Schnittstellen
  - Hierarchien: Enthalten-Sein-Beziehung von Bausteinen, Schichten, Zyklenfreiheit
  - Schemata: Anwendung von Entwurfs- und Architekturmustern

Quelle: (Lilienthal 2024)

Kopplung & Kohäsion (1)

- Kopplung
  - Maß der Abhängigkeit zwischen kooperierenden Bausteinen
  - Abhängigkeiten: Instanziierung, Operationsaufruf, Benachrichtigung, Daten, Laufzeitumgebung, zeitliche Reihenfolge etc.
  - Hohe Kopplung: Änderungen an einem Baustein führt zwangsläufig zu Änderungen an anderen Bausteinen
- Kohäsion
  - Maß für inneren, fachlichen Zusammenhang von Bausteinen
  - Geringe Kohäsion: Anzeichen, dass Code dupliziert und nicht wiederverwendet wurde [→ DRY-Prinzip]
- Ziel: schwache Kopplung & starke Kohäsion (Loose Coupling & Strong Cohesion)

Kopplung & Kohäsion (2)



SOLID-Prinzipien nach Robert C. Martin

SRP: Single Responsibility Principle

OCP: Open-Closed Principle

LSP: Liskov Substitution Principle

ISP: Interface Segregation Principle

DIP: Dependency Inversion Principle

Single Responsibility Principle

- Es sollte nur einen Grund geben, eine Klasse zu modifizieren
- Robert C. Martin: »Ein Modul sollte für einen, und nur einen, Akteur verantwortlich sein«
- Zusammenhang mit [→ Separation of Concerns]
- Dadurch: Funktionalität leichter zu beschreiben; verteilt implementierbar; verständlicher, fachlicher Zusammenhang [→ Kohäsion]; weniger Abhängigkeiten [→ Kopplung]; isoliertes Testing
- Beispiel: Verschlüsselung & Komprimierung trennen; Lesen von Daten & Verarbeitung von Daten trennen etc.
- Achtung: wird oft missverstanden; SRP bedeutet nicht, dass eine Klasse nur eine Aufgabe erfüllen soll

Open-Closed Principle (1)

- Auch gut geplante Systeme müssen erweitert werden
  - Neue Anforderungen, Auftraggebende entwickeln nur langsam Vorstellung des Zielsystems etc.
  - Veränderbarkeit und Anpassbarkeit ist wesentlich für die agile Softwareentwicklung
  - Clean Code: Konzept der Evolvierbarkeit
- Typisch: späte Features & Anpassungen werden immer teurer
  - Grund: wachsende Unordnung (Entropie) des Systems
  - Dadurch Komplexität: Entwickelnde müssen bestehenden Quellcode einer Klasse vor einer Änderung erst verstehen
  - Und: Möglichkeit des automatischen Testens nimmt ab
- OCP: System soll offen für Erweiterung, aber geschlossen gegenüber Modifikationen sein
- Wichtige treibende Kraft für gute Architektur

Quelle: (Meyer 1988)

Open-Closed Principle (2)

- Konsequenz
  - Bausteine sollen erweiterbar sein, ohne bestehenden Quellcode oder Schnittstellen modifizieren zu müssen (»pacta sunt servanda«)
  - In Komponentenhierarchie: Komponenten höherer Ebenen sollten geschützt sein vor Änderungen an untergeordneten Komponenten
- Typisches Missverständnis
  - Während Implementierungsphase (Iteration) natürlich Anpassung von Klassen möglich, z.B. Refaktorisierung; OCP gilt eher als Invariante am Anfang und am Ende einer Iteration
- Achtung: OCP steht oft im Widerspruch zum YAGNI-Prinzip; Erweiterbarkeit ist nur dort sinnvoll, wo auch der Bedarf nach einer Erweiterung abzusehen ist
- Strategy-Pattern [→ Exkurs D] ist gut geeignet, um OCP effizient umzusetzen

Liskov Substitution Principle (1)

- Wichtiges Merkmal von OOP ist Vererbung
  - Kann helfen, um Komplexität zu reduzieren
  - Funktioniert aber nur, wenn garantiert ist, dass Eigenschaften der Basisklasse in allen abgeleiteten Klassen gelten
  - Beispiel: mathematisch ist ein Quadrat ein spezielles Rechteck; aber sollte Quadrat eine abgeleitete Klasse der Basisklasse Rechteck sein?
- LSP: abgeleitete Klassen (Unterklasse) müssen grundsätzlich für ihre Basisklassen (Oberklasse) eingesetzt werden können
  - Wenn eine Operation ein Objekt verwendet in dem Glauben, es sei eine Instanz der Basisklasse, dann muss sich das Objekt auch so verhalten – auch wenn es tatsächlich Instanz einer abgeleiteten Klasse ist
- Beispiel (IRL): Subunternehmer im Auftrag von Amazon
  - Wenn nicht anders vereinbart, dann darf Sub nicht mehr erwarten als Amazon (Gebühren, Versandkosten, Anwesenheit etc.) und nicht weniger liefern (Produkte, Gewährleistung, Rückgaberecht etc.)

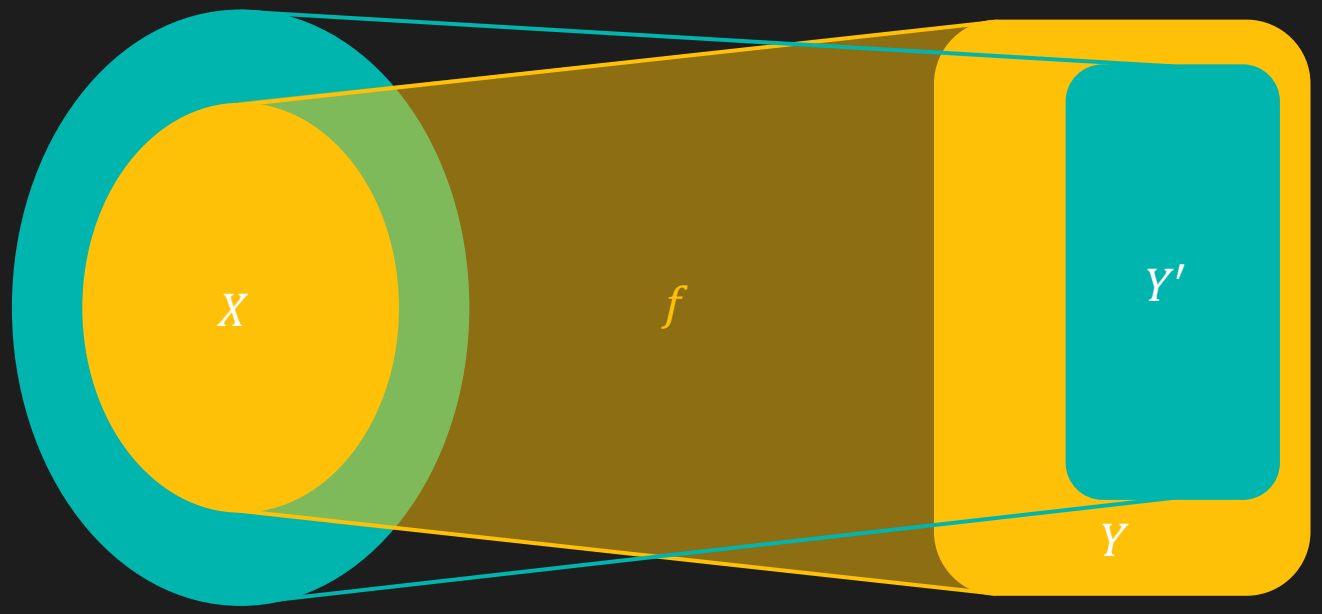
Quelle: (Liskov 1987)

Liskov Substitution Principle (2)

- Abgeleitete Klasse übernimmt alle Verpflichtungen & Zusicherungen der Basisklasse
- Abgeleitete Klassen dürfen nicht mehr erwarten als ihre Basisklasse (schwächere Vorbedingungen)
- Abgeleitete Klassen dürfen nicht weniger liefern als ihre Basisklasse (stärkere Nachbedingungen)
- Konsequenz für überschriebene Operationen
  - Übergabeparameter dürfen nur erweitert, aber nicht eingeschränkt werden (z.B. kleinere Wertebereiche), d.h. sie müssen kontravariant (entgegen Vererbungsrichtung) sein
  - Rückgabewerte dürfen nur eingeschränkt, aber nicht erweitert werden, d.h. sie müssen kovariant (mit Vererbungsrichtung) sein
- Das gilt auch für Ausnahmen (Exceptions)
  - Auch Form von Rückgabe: abgeleitete Klasse darf nur dort Ausnahmen auslösen, wo die Basisklasse das auch tut; nur Ausnahmen vom selben Typ oder Ableitungen davon

Kovarianz & Kontravarianz

- Substitutionsprinzip bei mathematischen Funktionen: um $f: X \to Y$ durch $f': X' \to Y'$ ersetzen zu können, muss gelten
  - $X \subseteq X'$: Definitionsbereich X' ist kontravariant (Obermenge) zu X
  - $Y' \subseteq Y$: Wertebereich Y' ist kovariant (Untermenge) zu Y
- Dann bleiben Ausdrücke wie $h(f(g(x))$ gültig, d.h. im jeweiligen Definitions-bereich der Funktionen

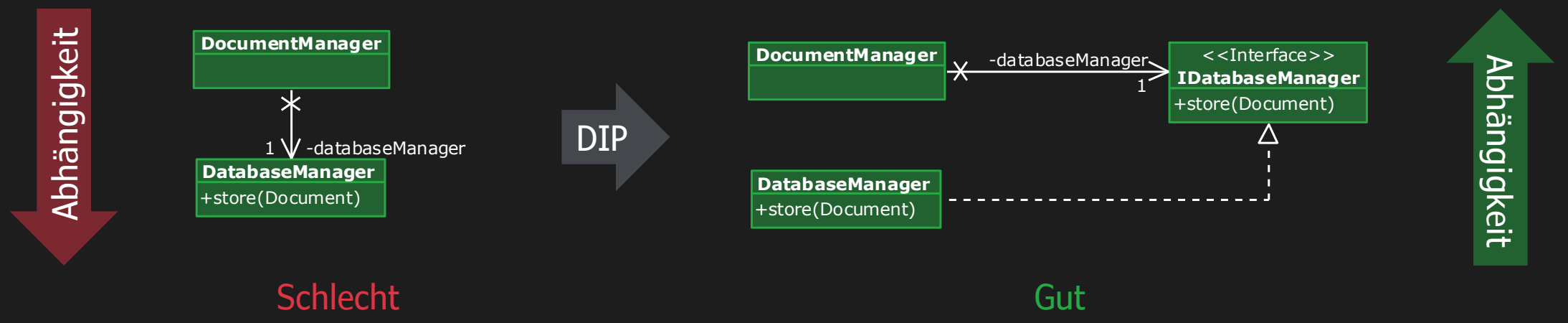


Interface Segregation Principle

- Konsumenten sollten nicht gezwungen sein, von Details abzuhängen, die sie nicht benötigen
  - Folgerung: Klassen sollten nicht Teile von Schnittstellen realisieren müssen, die sie nicht verwenden (hier: Schnittstelle als Interface bzw. abstrakte Klasse)
- ISP: verschiedene Verantwortungsbereiche einzelner Komponenten besitzen jeweils eine eigene Schnittstelle
- Verringert Kopplung & Abhängigkeiten; erhöht die Testbarkeit
- Schlecht
  - Eine große Universalschnittstelle
  - Technische Begründung von Schnittstellen (z.B. Datenbankzugriffe)
- Gut
  - Mehrere feingranulare, konsumentenspezifische Schnittstellen
  - Fachliche Begründung von Schnittstellen (z.B. Auftragsverwaltung)

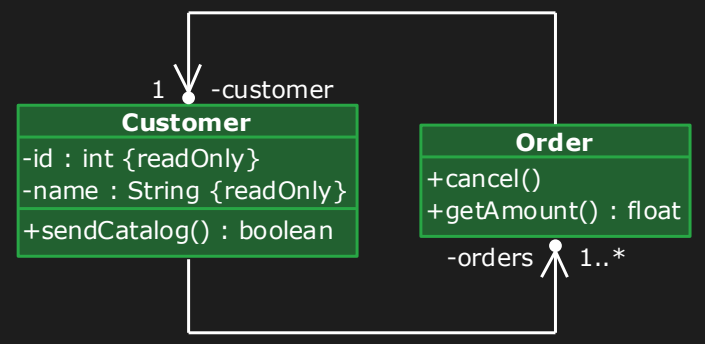
Dependency Inversion Principle

- Ziel: richtige Richtung der Abhängigkeiten
  - Konkrete von abstrakten Modulen; abgeleiteten Klassen von Basisklassen [→ kontravariant]
  - Siehe auch Layer-Architekturmuster
- Dependency Inversion: Umkehrung der Abhängigkeiten
  - Klassen hoher Ebenen sollen nicht von Klassen niedriger Ebenen abhängig sein; beide sollen von Abstraktionen abhängig sein
  - Abstraktionen sollen nicht von Details, sondern Details von Abstraktionen abhängen

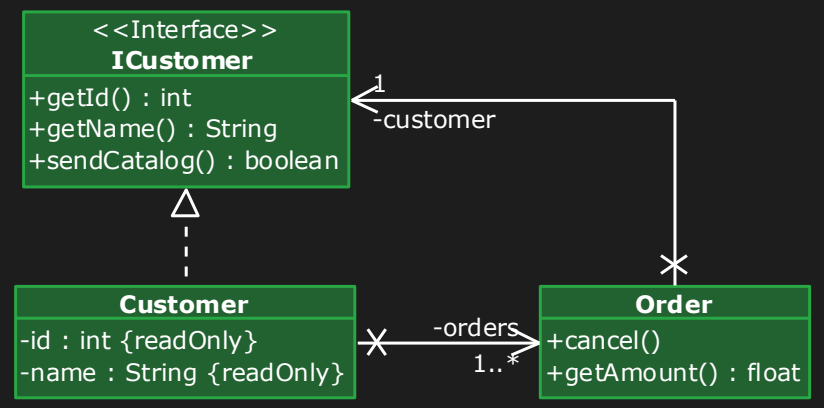


Zyklische Abhängigkeiten

- Dependency Inversion ist geeignet, um (problematische) zyklische Abhängigkeiten aufzulösen



Schlecht



Gut

Dependency Injection

- Durch DIP wird die Abhängigkeit umgedreht, aber sie bleibt eventuell in der Klasse versteckt (z.B. durch privates Attribut)
  - Intransparenz bzgl. der Abhängigkeiten
  - Ungünstig für Komponententests: die zu testende Klasse sollte von allen Abhängigkeiten isoliert werden können [→ Kapitel 6]
- Lösung durch explizite Angabe der Abhängigkeit mittels Dependency Injection (DI)
  - Constructor Injection Abhängigkeit wird als Parameter eines Konstruktors übergeben
  - Property Injection Abhängigkeit wird über einen Setter gesetzt
- Standard-Ansatz bei vielen Frameworks, z.B. Angular
- Verallgemeinerung: Inversion of Control (IoC)

Abschnitt 5.2

Clean Code & Anti-Pattern

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Clean Code Developer (CCD)

clean code developer

- »Mehr Professionalität in der Softwareentwicklung!« (Robert C. Martin)
- Grade: Schwarz » Rot » Orange » Gelb » Grün » Blau » Weiß
- Werte: Wandelbarkeit, Korrektheit, Produktionseffizienz, kontinuierliche Verbesserung

Prinzipielle Tugenden

- Schätze Variation (Value Variation)
- Tue nur das Nötigste (Do Only What's Neccessary)
- Isoliere Aspekte (Isolate Aspects)
- Minimiere Abhängigkeiten (Minimize Dependencies)
- Halte Versprechen ein (Honor Pledges)

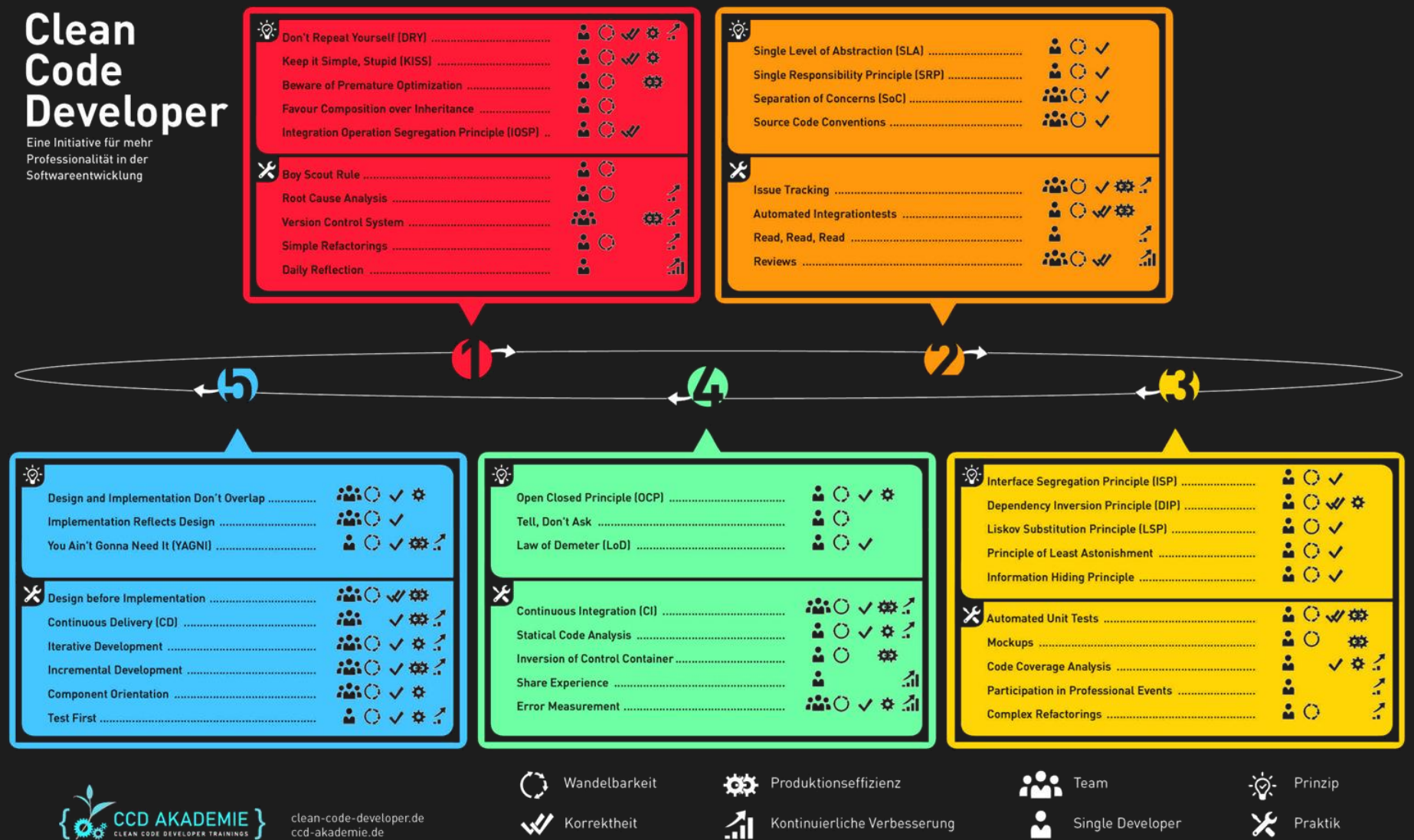
Praktische Tugenden

- Umarme Unsicherheit (Embrace Uncertainty)
- Fokussiere (Focus)
- Wertschätze Qualität (Value Quality)
- Mach fertig (Get Things Done)
- Halte Ordnung (Stay Clean)
- Bleib am Ball (Keep Moving)

Quelle: (Martin 2025)

Prinzipien & Praktiken

clean code developer



FH AACHEN UNIVERSITY OF APPLIED SCIENCES

CCD: Beispiel-Prinzipien

clean code developer

- Don’t Repeat Yourself (DRY)
  - »Jede Doppelung von Code oder Handgriffen leistet Inkonsistenzen und Fehlern Vorschub.«
- Keep It Simple, Stupid (KISS)
  - »Wer mehr tut als das Einfachste, lässt die Auftraggebenden warten und macht die Lösung unnötig kompliziert.«
- Beware of Premature Optimization
  - »Optimierungen kosten viel Aufwand. Spare wertvolle Ressourcen für das, was dem Kunden wirklich nützt.«
- Favour Composition over Inheritance (FCoI)
  - »Komposition fördert die lose Kopplung und die Testbarkeit eines Systems und ist oft flexibler.«
  - Komposition durch Referenz auf andere Klasse statt Vererbung
- Source Code Conventions (aka Coding Conventions)
  - »Code wird häufiger gelesen als geschrieben. Daher sind Konventionen wichtig, die ein schnelles Lesen und Erfassen des Codes unterstützen.«
- You Ain’t Gonna Need It (YAGNI)
  - »Dinge, die niemand braucht, haben keinen Wert. Verschwende an sie also keine Zeit.«

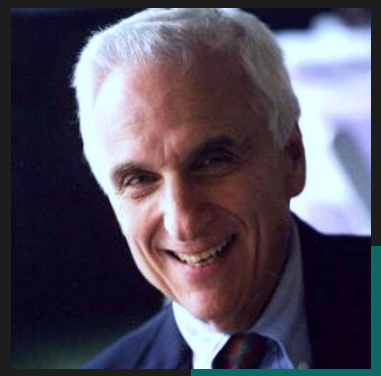
Coding Conventions

- Konventionen & Richtlinien zur Sicherstellung von qualitativ hochwertigem Code bei mehreren Entwickelnden
  - Organisation der Dateien
  - Sprechende Namen: Variablen, Klassen, Methoden etc.
  - Code-Layout, allgemeine Konstruktionen, Muster etc.
  - Kommentierung & Inline-Dokumentation
- Guidelines für viele Programmiersprachen verfügbar
- Sollte im Team / Unternehmen eingeführt und gelebt werden!
- Prüfung durch statische Codeanalyse
- Werkzeugunterstützung: Lint, Continuous Inspection, Code-Dokumentation (z.B. Javadoc, GhostDoc, Sphinx)

Anti-Pattern: Beispiele

- Gegenstück zu »guten« Entwurfsprinzipien; sollten vermieden werden!
- Golden Hammer: bekannter Lösungsansatz wird immer verwendet; »If all you have is a hammer, everything looks like a nail.« (A. Maslow)
- Not-Invented-Here-Syndrom: Software wird neu entwickelt, statt bestehende Lösungen oder Frameworks zu nutzen
- Big Ball of Mud: Software ohne erkennbare Architektur; monolithischer Code
- Onion Code: neue Funktionalität wird über alte Funktionalität gelegt
- Magic Values: hartkodierte Literale mit besonderer Bedeutung
- Body Ballooning: Erfolg gemessen an Zahl der Mitarbeitenden; Lösungen werden unnötig arbeitsintensiv
- Fear of Success: wegen Argwohn der Kollegen wird Innovation vermieden
- Single Head of Knowledge: Wissen wird eifersüchtig geheim gehalten
- Programmer Interrupt: viele Unterbrechungen durch wechselnde Projekte, Supportanfragen, Meetings, Unterstützung bei Angebots-erstellung etc.

Conway's Law (1)

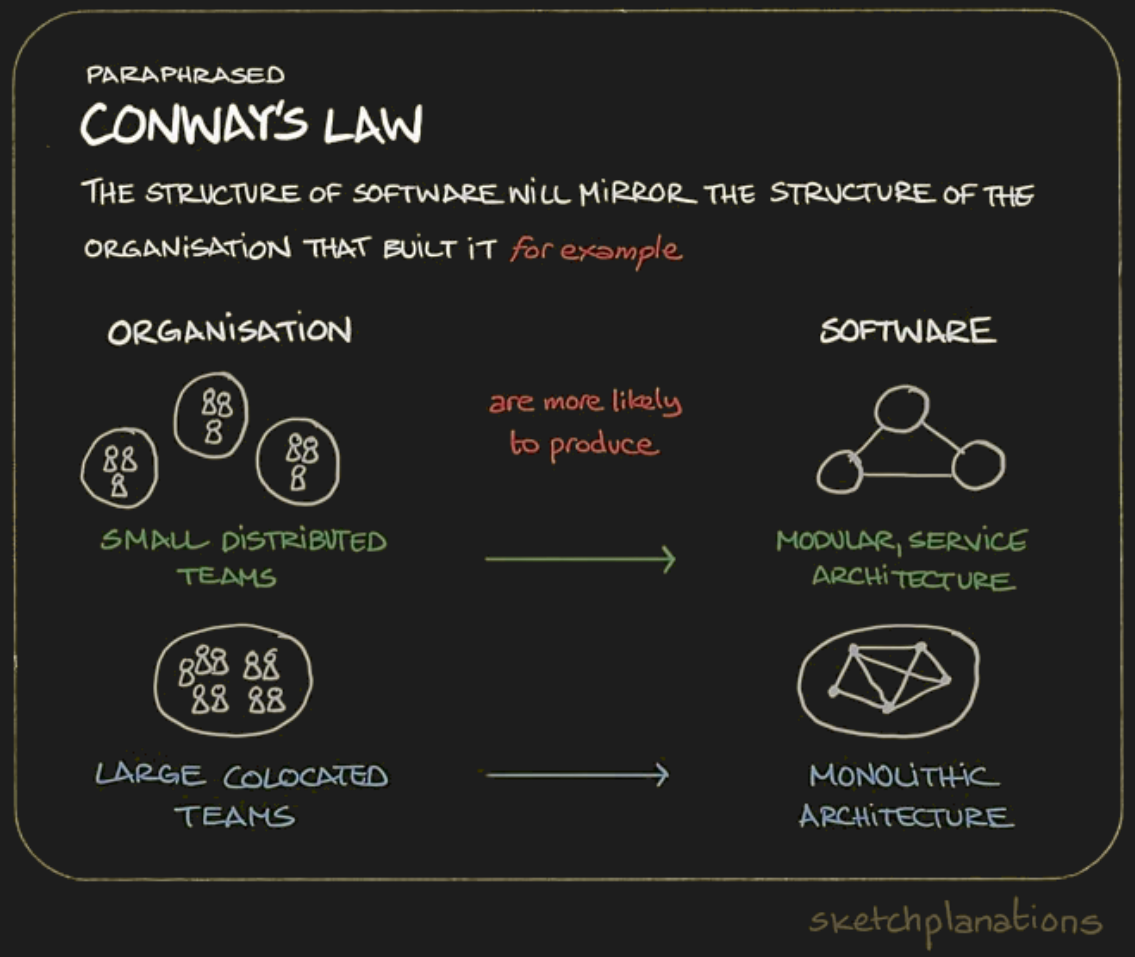


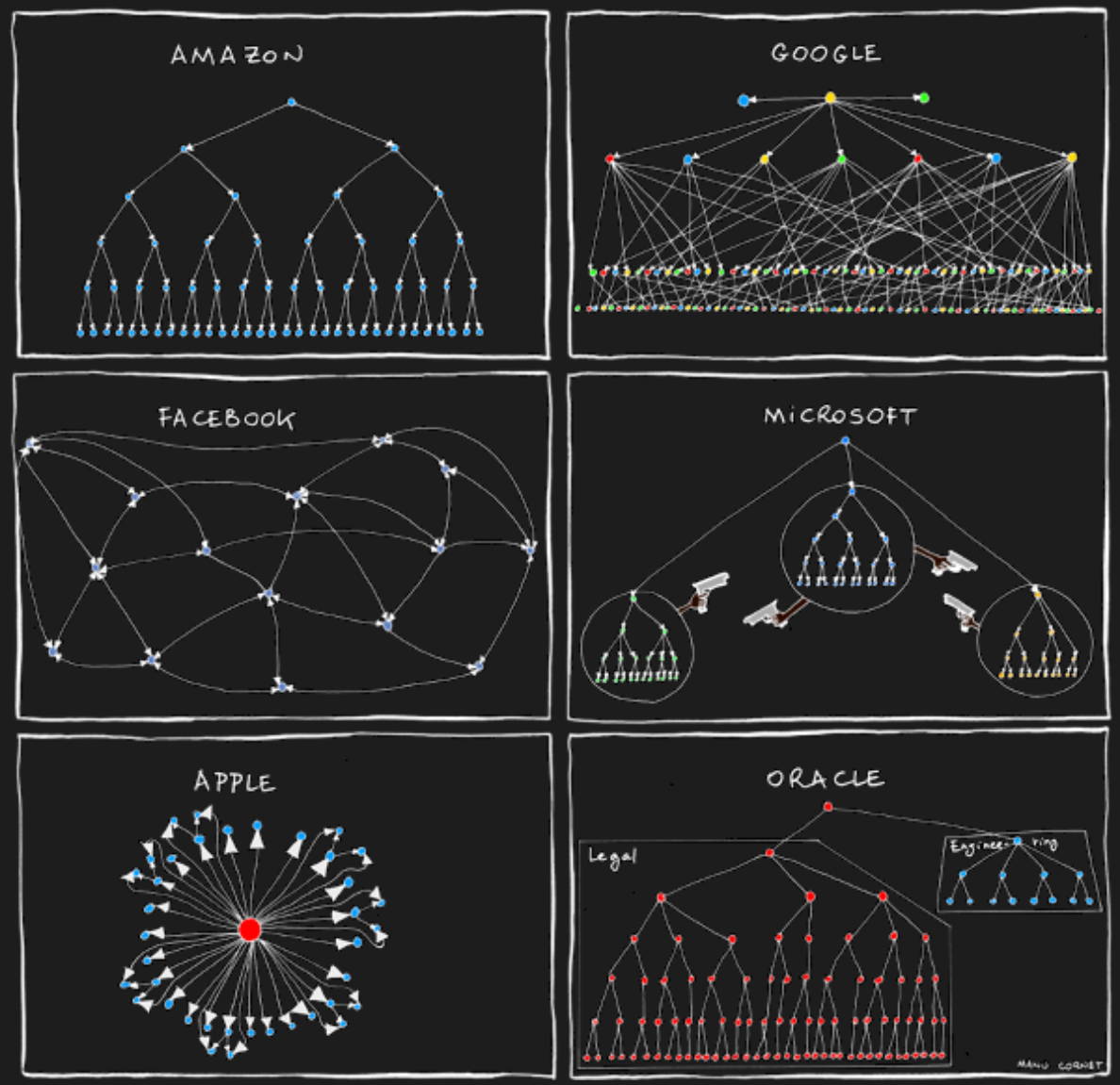
Melvin E. Conway

»Organizations which design systems [...] are constrained to produce designs which are copies of the communication structures of these organizations.«

Quelle: (Conway 1968)

Conway's Law (2)





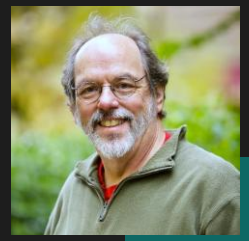
Abschnitt 5.3

Technische Schulden

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Technische Schulden

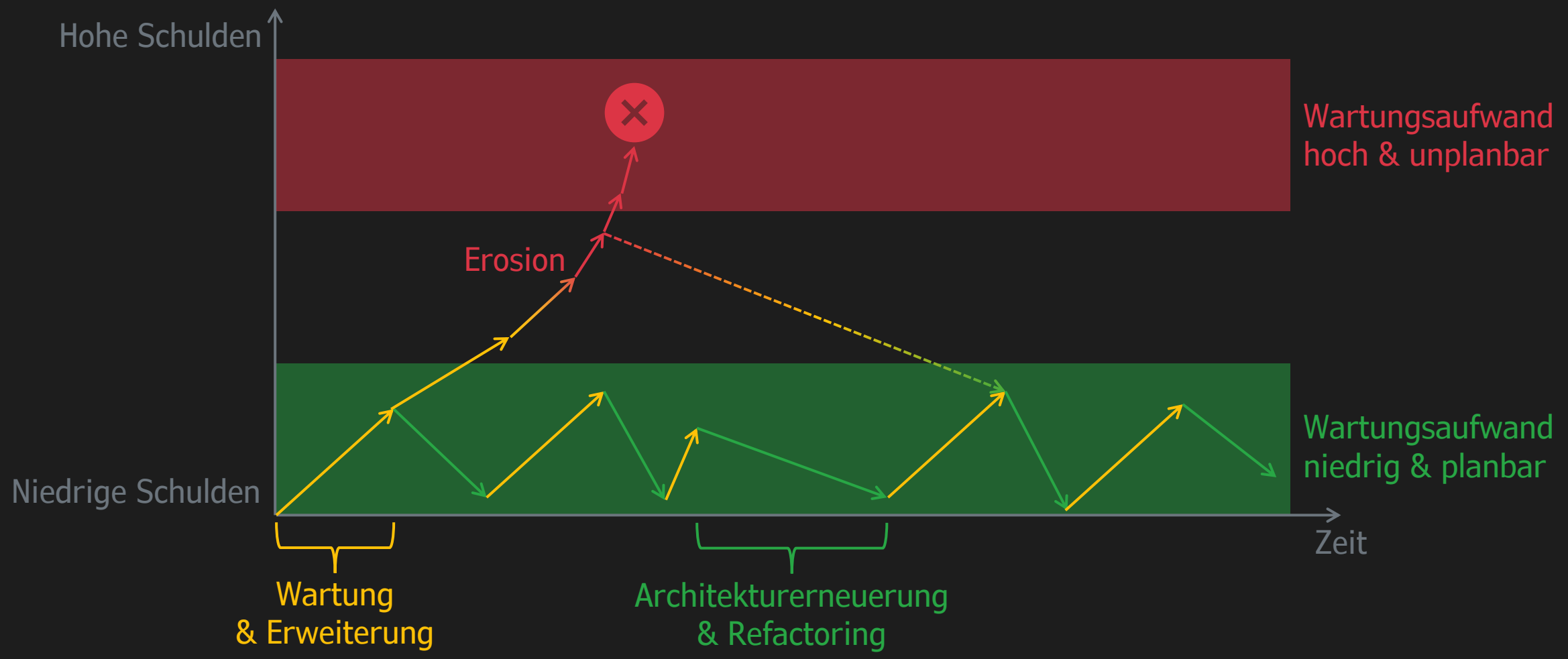
- Metapher für »Sanierungskosten« schlechter Software
- Ziel: mit betriebswirtschaftlichen Begriffen verdeutlichen, dass das Aufschieben von Qualitätsmaßnahmen die Softwareentwicklung verteuert
- Exakte Messung schwierig; Abschätzung durch Qualitätsmetriken
- Unterschied Anti-Pattern: Schulden können vorübergehend eingegangen werden



Ward Cunningham

»Software zu schreiben ist wie Schulden aufnehmen. Geringe Schulden aufzunehmen, beschleunigt die Softwareentwicklung, solange die Schulden rasch durch eine Neuimplementierung getilgt werden… Die Gefahr entsteht, wenn die Schulden nicht zurückgezahlt werden. Jede Minute, die man mit nicht ganz richtigem Code verbringt, zählt als Zins auf diese Schuld.« (Cunningham 1992)

Technische Schulden: Entwicklung



Quelle: (Lilienthal 2024)

Symptome der Architekturerosion

Rigidity

- System ist unflexibel gegenüber Änderungen
- Jede Änderung führt zu Kaskade an weiteren Änderungen
- Entwickler scheuen Änderungen

Fragility

- Änderungen führen zu überraschenden Fehlern ohne direkten Bezug zur eigentlichen Änderung

Immobility

- Wiederverwendung von Code ist kaum möglich, weil zu viele Nebeneffekte
- Isolieren von generischen Lösungen nicht möglich
- Code wird oft dupliziert

Viscosity

- Architekturerhaltende Anpassungen sind aufwendig
- Umgehungslösungen werden bevorzugt

Quelle: (Martin 2000)